

FOREST-FIRE & SMOKE RESPONSE SYSTEM: DEEP LEARNING-BASED APPROACHES FOR ANALYSIS OF CCTV IMAGES

Harina J M¹, Janani R², Leelavathi N V³, V. Priya^{4*}

^{123*}UG Scholar –Dept CSE, Grt Institute of Engineering and Technology, Tiruttani, India.

^{4*}Assistant Professor- Dept CSE, Grt Institute of Engineering and Technology, Tiruttani, India.

renujm3@gmail.com, janukavi2004@gmail.com, nvleela2004@gmail.com

^{4*}Corresponding Author: priya.v@grt.edu.in

<https://orcid.org/0009-0004-0367-4358>

Abstract - An effective forest-fire response is critical for minimizing the losses caused by forest fires. The purpose of this study is to construct a model for early fire detection in video and live feed response systems based on deep learning. Initially, we implement neural architecture search-based detection using the YOLO (You Only Look Once) model. Backbone networks play a crucial role in the application of deep learning-based models, as they have a significant impact on the performance of the model. To train and test our fire detection models, we utilize a large-scale fire dataset. We then compare the searched light-weight backbone with well-known backbones, such as YOLO4 Model and inception model. Additionally, our system captures essential metadata, including the date, time, latitude, and longitude of the fire detection area. This geographic information can be invaluable for the immediate repeated response and the coordination during firefighting efforts. Once a fire is detected, the system seamlessly transfers the collected data to an alert database, enabling immediate response actions.

Keywords: Private key, public key, CBI key and encrypting the data storage

1. INTRODUCTION

Forests contribute to significant ecological and economic functions in ecosystem. In addition, forests are important heritage sites for human beings. However, forest fires can cause tremendous damage to human life and property and adversely affect forest ecosystems in the long term. Recent advancements in intelligent fire detection and prediction systems have focused on integrating machine learning, deep learning, and real-time data analysis to enhance decision-making in disaster management. A hybrid model combining Cellular Automata and machine learning has been proposed to simulate fire spread direction, speed, and burned area, enabling more accurate forecasting and faster emergency responses [1]. In related work, deep learning models such as CNN-LSTM have been used to analyze weather-related events to predict power outages, aiding early detection and infrastructure protection during disasters [2]. Video-

based fire detection methods have also been improved using advanced object detection techniques, where enhanced YOLO-v4 and Vibe algorithms enable the system to detect fire within 16 frames, ensuring real-time monitoring and early alerts [3]. To further improve sensitivity and reliability, LSTM-based variational autoencoders have been applied to fire detection tasks, significantly reducing false positives and enhancing detection accuracy through the use of real-world datasets [4]. Additionally, unmanned aerial vehicles (UAVs) equipped with RGB and infrared imaging have been employed to capture early signs of wildfire in inaccessible areas, offering flexible and precise monitoring capabilities through deep learning-based image analysis [5]. With the advancement of deep learning in computer vision, traditional limitations in object detection and classification are being overcome. High-performance object detection models are broadly categorized into one-stage and two-stage models. One-stage models, like YOLO, SSD, and Retina Net, perform detection and classification simultaneously, offering speed but struggling with large-scale fire scenarios.

For instance, YOLOv3 achieved limited recognition and speed using UAV images, while an ensemble of YOLOv5 and Efficient Det improved precision but still lacked in wide-area detection. In contrast, two-stage models, such as Faster R-CNN, Dense Net, and Mask R-CNN, perform region proposal and classification in two separate steps, enhancing accuracy. Recent models like ATT Squeeze U-Net and Faster R-CNN (with backbones like Alex Net, VGG16, and ResNet101) have shown high accuracy (up to 93%) and better detection capabilities. Additionally, newer CNN architectures are being used to classify fire scenarios, though limitations in speed, scalability, and real-time application still exist.

2. YOLO MODEL

The YOLO (You Only Look Once) algorithm is a revolutionary breakthrough in the field of computer vision and real-time object detection. Unlike traditional object detection techniques that operate in multiple stages—first generating region proposals



and then classifying them—YOLO simplifies the entire process into a single-stage pipeline. It is built upon a deep learning-based convolutional neural network (CNN) architecture that enables the model to detect and classify objects within an image in just one forward pass. This unique approach significantly reduces the computational cost and increases detection speed, making it highly efficient and suitable for real-time applications. YOLO works by dividing the input image and make it qualified images into a grid and allowing each grid cell to predict bounding boxes along with confidence scores and class probabilities. This enables the model to detect multiple objects within an image in just one forward pass from these data sets by using its various positions and sizes simultaneously. Its speed and accuracy make it ideal for high-demand environments such as autonomous driving, real-time.

3.RELATED WORK

Fire detection technologies have advanced rapidly with the integration of deep learning, UAV platforms, and multimodal sensing. Traditional approaches, such as satellite imaging and ground-based surveillance, often suffer from high latency and limited real-time effectiveness, especially under conditions like electric field interference or low-visibility environments. An improved fire detection system has been proposed using an enhanced YOLOv4 model combined with the Vibe algorithm. By incorporating a simplified weighted Bi-directional Feature Pyramid Network (Bi-FPN), this approach enables robust multi-scale feature fusion and accurate detection of dynamic fire behaviours. The model demonstrated high accuracy (98.9%) and a low false alarm rate (2.2%) even in electrically noisy conditions [1],[8].

To address the need for high-quality aerial data, a drone-based dataset was introduced, featuring synchronized RGB and infrared videos from prescribed burns. This dataset also includes geo-referenced point clouds, maps, weather data, and fire plans, enabling precise pixel-level fire and smoke segmentation for training advanced deep learning models [2]. For fire spread prediction, a hybrid model integrating Cellular Automata and machine learning has been developed. This system includes two core components: a fire spread process prediction module for forecasting direction and speed, and a results prediction module for estimating the affected area. Validated on real-world case studies, this model has shown superior performance compared to traditional simulators like Far site and Prometheus [3]. In the context of infrastructure protection, a CNN-LSTM model was designed to predict fire-induced power outages. By leveraging

real-time meteorological data from ground-based weather stations and applying preprocessing techniques such as deduplication and anomaly detection, this model significantly improves outage prediction accuracy [4],[5]. Further developments in fire detection include a YOLO-based segmentation framework tailored for detecting fire at the pixel level, improving detection under complex visual conditions [7]. Lightweight CNN architectures have also been introduced for fire classification in surveillance systems, balancing speed and accuracy for real-time deployment [9]. Transformer-based architectures have been applied to satellite imagery for fire segmentation, using tokenized time-series data to enhance detection accuracy in dynamic environments [1]. Additionally, a Deep LSTM Variational Autoencoder has been proposed to improve the sensitivity and reliability of fire detection systems under uncertain and noisy conditions [6].

4.PROPOSED SYSTEM

In this research initiative, we propose a comprehensive and intelligent forest-fire response system that leverages state-of-the-art deep learning techniques to address two critical challenges: rapid fire detection and precise geospatial localization. The core of our system is built around real-time processing of live surveillance camera feeds, enabling timely detection and response to emerging fire threats. For efficient and accurate object detection, we integrate the powerful YOLO (You Only Look Once) model, known for its high-speed, single-shot detection capabilities that make it exceptionally suitable for real-time applications in dynamic environments.

Complementing YOLO, our framework incorporates a robust image classification and detection algorithm grounded in the Inception architecture. This deep convolutional neural network adds an additional layer of detection precision by effectively learning complex fire patterns across multiple scales, enhancing the overall system accuracy. To further improve detection reliability, especially in cluttered or visually noisy forest environments, we implement spatial segmentation techniques using deep learning. These methods enable our system to isolate fire regions within each video frame, distinguishing them from non-relevant background elements such as sunlight, fog, or moving animals. To enhance fire detection accuracy, the proposed system incorporates motion compensation techniques to handle camera movement and environmental instability, ensuring temporal coherence in dynamic video feeds.

By integrating real-time object detection, spatial segmentation, and motion-aware processing, the model exhibits high adaptability, precision, and



scalability. This makes it well-suited for diverse forest monitoring scenarios and highlights its potential for integration into broader wildfire management systems, supporting early warnings and proactive disaster response efforts. Additionally, implementing edge computing will enable faster on-site processing, reducing the latency in alert generation. The use of advanced deep learning models such as Transformer-based architectures and reinforcement learning can further boost detection accuracy and adaptive learning over time. Lastly, integrating this system with centralized emergency response platforms can automate alerts, enable quicker deployment of firefighting resources in general, case can be defined as a course or principle of action adopted or proposed by an organization or individual.

5.1 IMAGE PREPROCESSING:

This process begins by ingesting video streams, which often consist of a sequence of frames captured by cameras or video. The first step involves frame extraction, where individual frames are separated from the video stream. These frames input data for the YOLO model, and their high- resolution nature can significantly impact detection accuracy. This process begins by ingesting video streams, which often consist of a sequence of frames captured by cameras or drones.

The first step involves frame extraction, where individual frames are separated from the video stream. These frames serve as the input data for the YOLO model, and their high-resolution nature can significantly impact detection accuracy. Data augmentation may be employed during preprocessing to diversify the dataset and improve the model's generalization. Techniques like random cropping, rotation, and brightness adjustments introduce variability into the training data, making the model more robust to real-world scenarios and environmental conditions

5.2 FEATURE EXTRACTION:

These features encapsulate critical information regarding fire-related characteristics, including color, texture, and shape, among others. Color-based features also play a significant role in forest fire detection. The feature extraction enhance the color of the image and increase the flames often exhibit distinct color signatures, such as variations in brightness and hue. By quantifying these color variations and incorporating them as features, the model can distinguish flames from non-fire elements effectively. These specialized neural networks are designed to automatically learn and extract hierarchical features from images. Through series of convolutional layers, CNNs identify low-level features like edges, corners, and textures and

progressively built high-level features that represent complex patterns, such as flames, or fire-related structures. These learned features become the foundation for subsequent fire. flames, or fire-related structures.. Flames often exhibit distinct color signatures, such as variations in brightness and hue. By quantifying these color variations and incorporating them as features, the model can distinguish flames from non-fire elements effectively.

5.3 YOLO MODEL:

The YOLO (You Only Look Once) algorithm represents a groundbreaking advancement in computer vision and object detection. At its core, YOLO is a deep learning-based convolutional neural network (CNN) architecture designed to excel in real-time object detection tasks. Unlike traditional object detection methods that require multiple passes through the network, YOLO adopts a unique approach by processing the entire image or frame in a single forward pass. This efficiency is achieved through a grid-based system that divides the input into a fixed number of cells and associates each cell with bounding box predictions and the class probabilities used in the model.

5.4. DETECTION AND ALERT SYSTEM

Leveraging the YOLO (You Only Look Once) model for fire detection in both video streams and live camera feeds represents a significant advancement in fire management and safety. This application harnesses the power of deep learning and real-time object detection to identify and respond to fire incidents swiftly and effectively. In this context, the YOLO model operates by processing video frames in real-time, utilizing deep convolutional neural network architecture to simultaneously recognize and locate fire within the frames. This approach ensures that fire incidents can be detected as soon as they appear in the video feed, enabling rapid response. One of the key advantages of using YOLO for fire detection is its speed and accuracy. When a fire is detected, whether through visual analysis, the system immediately triggers an alert mechanism. This alert includes crucial details such as the image capturing the fire event, the precise location coordinates (latitude and longitude), and a timestamp.

6.ALGORITHM:

YOLO was proposed by Joseph Redmond *et al.* in 2015. It was proposed to deal with the problems faced by the object recognition models at that time and it sends the collected information to the alert data base, Fast R-CNN is one of the state-of-the-art models at that time but it has its own challenges such as this network cannot be used in real-time, because it

Diagram illustrating a 3D U-Net architecture for medical image segmentation. The network consists of an encoder (left) and a decoder (right), connected by skip connections.

Encoder (Left):

- Input: 448x448x3
- Layer 1: Conv. Layer 7x7x64x2
- Layer 2: Maxpool. Layer 2x2x2
- Layer 3: Conv. Layer 3x3x96
- Layer 4: Maxpool. Layer 2x2x2
- Layer 5: Conv. Layer 3x3x128
- Layer 6: Maxpool. Layer 2x2x2
- Layer 7: Conv. Layer 3x3x160
- Layer 8: Maxpool. Layer 2x2x2
- Layer 9: Conv. Layer 3x3x192
- Layer 10: Maxpool. Layer 2x2x2
- Layer 11: Conv. Layer 3x3x256
- Layer 12: Maxpool. Layer 2x2x2
- Layer 13: Conv. Layer 3x3x320
- Layer 14: Maxpool. Layer 2x2x2
- Layer 15: Conv. Layer 3x3x384
- Layer 16: Maxpool. Layer 2x2x2
- Layer 17: Conv. Layer 3x3x448
- Layer 18: Maxpool. Layer 2x2x2
- Layer 19: Conv. Layer 3x3x512
- Layer 20: Maxpool. Layer 2x2x2
- Layer 21: Conv. Layer 3x3x576
- Layer 22: Maxpool. Layer 2x2x2
- Layer 23: Conv. Layer 3x3x640
- Layer 24: Maxpool. Layer 2x2x2
- Layer 25: Conv. Layer 3x3x704
- Layer 26: Maxpool. Layer 2x2x2
- Layer 27: Conv. Layer 3x3x768
- Layer 28: Maxpool. Layer 2x2x2
- Layer 29: Conv. Layer 3x3x832
- Layer 30: Maxpool. Layer 2x2x2
- Layer 31: Conv. Layer 3x3x896
- Layer 32: Maxpool. Layer 2x2x2
- Layer 33: Conv. Layer 3x3x960
- Layer 34: Maxpool. Layer 2x2x2
- Layer 35: Conv. Layer 3x3x1024
- Layer 36: Maxpool. Layer 2x2x2
- Layer 37: Conv. Layer 3x3x1088
- Layer 38: Maxpool. Layer 2x2x2
- Layer 39: Conv. Layer 3x3x1152
- Layer 40: Maxpool. Layer 2x2x2
- Layer 41: Conv. Layer 3x3x1216
- Layer 42: Maxpool. Layer 2x2x2
- Layer 43: Conv. Layer 3x3x1280
- Layer 44: Maxpool. Layer 2x2x2
- Layer 45: Conv. Layer 3x3x1344
- Layer 46: Maxpool. Layer 2x2x2
- Layer 47: Conv. Layer 3x3x1408
- Layer 48: Maxpool. Layer 2x2x2
- Layer 49: Conv. Layer 3x3x1472
- Layer 50: Maxpool. Layer 2x2x2
- Layer 51: Conv. Layer 3x3x1536
- Layer 52: Maxpool. Layer 2x2x2
- Layer 53: Conv. Layer 3x3x1600
- Layer 54: Maxpool. Layer 2x2x2
- Layer 55: Conv. Layer 3x3x1664
- Layer 56: Maxpool. Layer 2x2x2
- Layer 57: Conv. Layer 3x3x1728
- Layer 58: Maxpool. Layer 2x2x2
- Layer 59: Conv. Layer 3x3x1792
- Layer 60: Maxpool. Layer 2x2x2
- Layer 61: Conv. Layer 3x3x1856
- Layer 62: Maxpool. Layer 2x2x2
- Layer 63: Conv. Layer 3x3x1920
- Layer 64: Maxpool. Layer 2x2x2
- Layer 65: Conv. Layer 3x3x1984
- Layer 66: Maxpool. Layer 2x2x2
- Layer 67: Conv. Layer 3x3x2048
- Layer 68: Maxpool. Layer 2x2x2
- Layer 69: Conv. Layer 3x3x2112
- Layer 70: Maxpool. Layer 2x2x2
- Layer 71: Conv. Layer 3x3x2176
- Layer 72: Maxpool. Layer 2x2x2
- Layer 73: Conv. Layer 3x3x2240
- Layer 74: Maxpool. Layer 2x2x2
- Layer 75: Conv. Layer 3x3x2304
- Layer 76: Maxpool. Layer 2x2x2
- Layer 77: Conv. Layer 3x3x2368
- Layer 78: Maxpool. Layer 2x2x2
- Layer 79: Conv. Layer 3x3x2432
- Layer 80: Maxpool. Layer 2x2x2
- Layer 81: Conv. Layer 3x3x2496
- Layer 82: Maxpool. Layer 2x2x2
- Layer 83: Conv. Layer 3x3x2560
- Layer 84: Maxpool. Layer 2x2x2
- Layer 85: Conv. Layer 3x3x2624
- Layer 86: Maxpool. Layer 2x2x2
- Layer 87: Conv. Layer 3x3x2688
- Layer 88: Maxpool. Layer 2x2x2
- Layer 89: Conv. Layer 3x3x2752
- Layer 90: Maxpool. Layer 2x2x2
- Layer 91: Conv. Layer 3x3x2816
- Layer 92: Maxpool. Layer 2x2x2
- Layer 93: Conv. Layer 3x3x2880
- Layer 94: Maxpool. Layer 2x2x2
- Layer 95: Conv. Layer 3x3x2944
- Layer 96: Maxpool. Layer 2x2x2
- Layer 97: Conv. Layer 3x3x3008
- Layer 98: Maxpool. Layer 2x2x2
- Layer 99: Conv. Layer 3x3x3072
- Layer 100: Maxpool. Layer 2x2x2
- Layer 101: Conv. Layer 3x3x3136
- Layer 102: Maxpool. Layer 2x2x2
- Layer 103: Conv. Layer 3x3x3200
- Layer 104: Maxpool. Layer 2x2x2
- Layer 105: Conv. Layer 3x3x3264
- Layer 106: Maxpool. Layer 2x2x2
- Layer 107: Conv. Layer 3x3x3328
- Layer 108: Maxpool. Layer 2x2x2
- Layer 109: Conv. Layer 3x3x3392
- Layer 110: Maxpool. Layer 2x2x2
- Layer 111: Conv. Layer 3x3x3456
- Layer 112: Maxpool. Layer 2x2x2
- Layer 113: Conv. Layer 3x3x3520
- Layer 114: Maxpool. Layer 2x2x2
- Layer 115: Conv. Layer 3x3x3584
- Layer 116: Maxpool. Layer 2x2x2
- Layer 117: Conv. Layer 3x3x3648
- Layer 118: Maxpool. Layer 2x2x2
- Layer 119: Conv. Layer 3x3x3712
- Layer 120: Maxpool. Layer 2x2x2
- Layer 121: Conv. Layer 3x3x3776
- Layer 122: Maxpool. Layer 2x2x2
- Layer 123: Conv. Layer 3x3x3840
- Layer 124: Maxpool. Layer 2x2x2
- Layer 125: Conv. Layer 3x3x3904
- Layer 126: Maxpool. Layer 2x2x2
- Layer 127: Conv. Layer 3x3x3968
- Layer 128: Maxpool. Layer 2x2x2
- Layer 129: Conv. Layer 3x3x4032
- Layer 130: Maxpool. Layer 2x2x2
- Layer 131: Conv. Layer 3x3x4096
- Layer 132: Maxpool. Layer 2x2x2
- Layer 133: Conv. Layer 3x3x4160
- Layer 134: Maxpool. Layer 2x2x2
- Layer 135: Conv. Layer 3x3x4224
- Layer 136: Maxpool. Layer 2x2x2
- Layer 137: Conv. Layer 3x3x4288
- Layer 138: Maxpool. Layer 2x2x2
- Layer 139: Conv. Layer 3x3x4352
- Layer 140: Maxpool. Layer 2x2x2
- Layer 141: Conv. Layer 3x3x4416
- Layer 142: Maxpool. Layer 2x2x2
- Layer 143: Conv. Layer 3x3x4480
- Layer 144: Maxpool. Layer 2x2x2
- Layer 145: Conv. Layer 3x3x4544
- Layer 146: Maxpool. Layer 2x2x2
- Layer 147: Conv. Layer 3x3x4608
- Layer 148: Maxpool. Layer 2x2x2
- Layer 149: Conv. Layer 3x3x4672
- Layer 150: Maxpool. Layer 2x2x2
- Layer 151: Conv. Layer 3x3x4736
- Layer 152: Maxpool. Layer 2x2x2
- Layer 153: Conv. Layer 3x3x4800
- Layer 154: Maxpool. Layer 2x2x2
- Layer 155: Conv. Layer 3x3x4864
- Layer 156: Maxpool. Layer 2x2x2
- Layer 157: Conv. Layer 3x3x4928
- Layer 158: Maxpool. Layer 2x2x2
- Layer 159: Conv. Layer 3x3x4992
- Layer 160: Maxpool. Layer 2x2x2
- Layer 161: Conv. Layer 3x3x5056
- Layer 162: Maxpool. Layer 2x2x2
- Layer 163: Conv. Layer 3x3x5120
- Layer 164: Maxpool. Layer 2x2x2
- Layer 165: Conv. Layer 3x3x5184
- Layer 166: Maxpool. Layer 2x2x2
- Layer 167: Conv. Layer 3x3x5248
- Layer 168: Maxpool. Layer 2x2x2
- Layer 169: Conv. Layer 3x3x5312
- Layer 170: Maxpool. Layer 2x2x2
- Layer 171: Conv. Layer 3x3x5376
- Layer 172: Maxpool. Layer 2x2x2
- Layer 173: Conv. Layer 3x3x5440
- Layer 174: Maxpool. Layer 2x2x2
- Layer 175: Conv. Layer 3x3x5504

YOLO can simultaneously predict multiple objects' classes and their corresponding bounding box coordinates with impressive speed, making it well-suited for applications like autonomous driving. This architecture takes an image as input and resizes it to 448×448 by keeping the aspect ratio same and performing padding. This image is then passed in the CNN network. This model has 24 convolution layers, 4 max- pooling layers followed by 2 fully connected layers. For the reduction of the number of layers (Channels), we use 1×1 convolution that is followed by 3×3 convolution. Notice that the last layer of YOLOv1 predicts a cuboidal output. This is done by generating (1, 1470) from final fully connected layer and reshaping it to size (7, 7, 30). This architecture uses Leaky ReLU as its activation function in whole architecture except the last layer where it uses linear activation function. The definition of Leaky ReLU can be found [here](#). Batch normalization also helps to regularize the model. Dropout technique is also used to prevent overfitting.

This model is trained on the *ImageNet-1000* dataset. The model is trained over a week and achieve top-5 accuracy of 88% on *ImageNet 2012* validation which is comparable to Google Net (2014 ILSVRC winner), the state of the art model at that time. Fast

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

8.DETECTION

The first two parts of the above loss equation be able to consider the deviation in terms of the size of the bounding box. For small bounding boxes, the little deviation should be more important as compared to large boxes. There are three terms in classification loss, the first term calculates the sum-squared error between the predicted confidence score that whether the object present or not and the ground truth for each bounding box in each cell. Similarly, The second term calculates the mean-squared sum of cells that do not contain any bounding box, and a regularization parameter is used to make this loss small. The third term calculates the sum-squared error of the classes belongs to these grid cells. Case -based

management of a multi-user workstation typically includes setting individual cases for such things as access to files or applications, various levels of access (Granularity), the appearance and makeup of individual users. The admin is the authorized role player to define and view the individual case base are viewed on the basis of category.

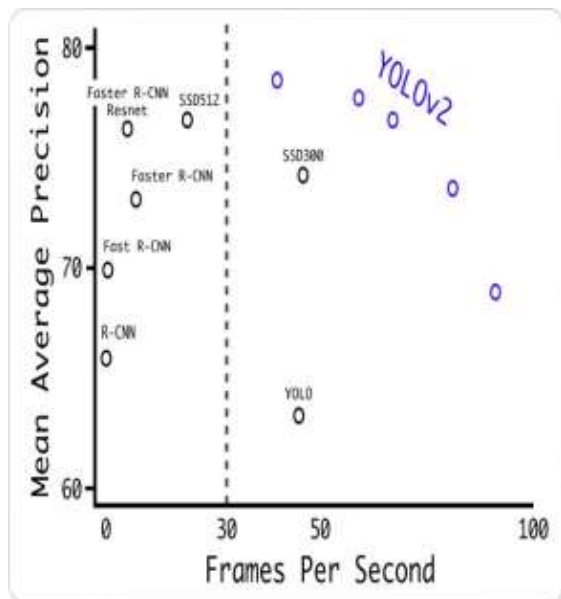


Fig.8.1. Mean average

9.EXPERIMENTAL RESULTS

This result discusses about the implementation of the policy based security for various cases are identified and the below Fig. 9.1., Fig. 9.2. and Fig. 9.3 Shows the implementation of admin policy based on the proposed methodology

```
Python 3.10.9 (tags/v3.10.9:aa5f517, Oct 11 2022, 16:50:30) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\project2k25\FIRE DETECTION\FIRE DETECTION\main.py =====
pygame 2.6.1 (SDL 2.28.4, Python 3.10.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
YOLOv5 2023-1-12 python=3.10.9 torch=2.0.0+cpu CPU

Fusing layers...
Model summary: 214 layers, 1025023 parameters, 0 gradients, 14.8 GFLOPs
Adding AutoShape...
* Serving Flask app 'main'
* Debug mode: off
[[31m[!WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.]]m
* Running on http://127.0.0.1:6060
[[33mPress CTRL+C to quit]]m
```

Fig.9.1. Shows the Execution



Fig.9.2. Fire Detection and its Home Page



Fig.9.3. Fire Identification

10.CONCLUSION & FUTURE WORK

In conclusion, the forest fire detection project, built upon the YOLO (You Only Look Once) model, has successfully addressed the critical need for early and efficient detection of wildfires. This project offers a multifaceted approach to fire management, combining cutting-edge technology with real-time monitoring and alerting systems. By utilizing deep learning and computer vision techniques,

it can swiftly and accurately identify fire incidents within video streams and live camera feeds. The integration of a database to store critical data such as fire images, timestamps, and location coordinates further enhances the project's capabilities. This database serves as a valuable repository for incident records, enabling post-incident analysis and research, trend identification, and data-driven decision-making.

To further improve the effectiveness of the forest fire detection system, future work could focus on integrating drone-based aerial surveillance with the YOLO model to expand coverage in remote and inaccessible areas. Additionally, incorporating weather data and environmental sensors (such as temperature, humidity, and wind speed) can enable

the development of predictive models that assess fire risk and potential spread. Implementing edge computing can also minimize latency by processing data closer to the source, allowing for faster response times in critical situations.

Time-Series in Satellite Image Segmentation With Transformer Network for Active Fire Detection," in *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1-13, 2023, Art no. 4405513, doi: 10.1109/TGRS.2023.3287498

REFERENCE

- [1] Shuai Zhao 1, Boyun Liu 1, Zheng Chi 1, Taiwei Li2, And Shengnan Li, "Characteristics Based Fire Detection System Under the Effect of Electric Fields With Improved Yolo-v4 and ViBe," *IEEE*, vol. 10, pp. 899-909, July. 2022.
- [2] X. Chen *et al.*, "Wildland Fire Detection and Monitoring Using a Drone-Collected RGB/IR Image Dataset," in *IEEE Access*, vol. 10, pp. 121301-121317, 2022, doi: 10.1109/ACCESS.2022.3222805.
- [3] X. Sun *et al.*, "A Forest Fire Prediction Model Based on Cellular Automata and Machine Learning," in *IEEE Access*, vol. 12, pp. 55389-55403, 2024, doi: 10.1109/ACCESS.2024.3389035.
- [4] W. Huang, W. Zhang, Q. Chen, B. Feng and X. Li, "Prediction Algorithm for Power Outage Areas of Affected Customers Based on CNN-LSTM," in *IEEE Access*, vol. 12, pp. 15007-15015, 2024, doi:
- [5] W. Huang, W. Zhang, Q. Chen, B. Feng and X. Li, "Prediction Algorithm for Power Outage Areas of Affected Customers Based on CNN-LSTM," in *IEEE Access*, vol. 12, pp. 15007-15015, 2024, doi:
- [6] Z. Xu, Y. Guo and J. H. Saleh, "Advances Toward the Next Generation Fire Detection: Deep LSTM Variational Autoencoder for Improved Sensitivity and Reliability," in *IEEE Access*, vol. 9, pp. 30636-30653, 2021, doi
- [7] X. Cao, Y. Su, X. Geng and Y. Wang, "YOLO-SF: YOLO for Fire Segmentation Detection," in *IEEE Access*, vol. 11, pp. 111079-111092, 2023, doi: 10.1109/ACCESS.2023.3322143.
- [8] S. Zhao, B. Liu, Z. Chi, T. Li and S. Li, "Characteristics Based Fire Detection System Under the Effect of Electric Fields With Improved Yolo-v4 and ViBe," in *IEEE Access*, vol. 10, pp. 81899-81909, 2022, doi:
- [9] D. -L. Nguyen, M. D. Putro and K. -H. Jo, "Lightweight Convolutional Neural Network for Fire Classification in Surveillance System," in *IEEE Access*, vol. 11, pp. 101604-101615, 2023, doi: 10.1109/ACCESS.2023.3305455.
- [10] Y. Zhao, Y. Ban and J. Sullivan, "Tokenized